

Assignment 5

Join Tuning

Database Tuning

Group Name (A18)
Krajinovic Ante, 2244629

May 26, 2026

Experimental Setup

Describe your experimental setup in a few lines.

Für die Implementierung wurden Java, Visual Studio Code und PostgreSQL (pgAdmin) verwendet. Die Verbindung zur Datenbank erfolgte über einen JDBC-Driver. Mittels CREATE TABLE wurden die Tabellen auth und publ erstellt und anschließend über Batch-Übertragung mit PreparedStatement aus TSV-Dateien befüllt. Die Ausführungszeiten sowie die Query-Pläne wurden mit EXPLAIN ANALYZE untersucht.

Join Strategies Proposed by System

Response times

Indexes	Join Strategy Q1	Join Strategy Q2
no index	Hash Join/318.660 ms	Hash Join/129.636 ms
unique non-clustering on Publ.pubID	Hash Join/274.159 ms	Nested Loop/136.852 ms
clustering on Publ.pubID and Auth.pubID	Hash Join/239.347 ms	Nested Loop/133.889 ms

Query plans

no index (Q1/Q2):

```
EXPLAIN ANALYZE SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
EXPLAIN ANALYZE SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

```
"Gather (cost=1668.87..50913.81 rows=45245 width=144) (actual time=6.050..318.263 rows=2324.00 loops=1)
"  Workers Planned: 2"
"  Workers Launched: 2"
"  Buffers: shared hit=15893 read=12004"
"  -> Hash Join (cost=668.87..45389.31 rows=18852 width=144) (actual time=114.161..245.728 rows=18852 loops=1)
"    Hash Cond: ((auth.pubid)::text = (publ.pubid)::text)"
"    Buffers: shared hit=15893 read=12004"
"    -> Parallel Seq Scan on auth (cost=0.00..39695.67 rows=1289667 width=38) (actual time=113.954..245.728 rows=1289667 loops=1)
"      Buffers: shared hit=14795 read=12004"
"    -> Hash (cost=500.61..500.61 rows=13461 width=145) (actual time=5.534..5.534 rows=13461 loops=1)
"      Buckets: 16384 Batches: 1 Memory Usage: 2468kB"
"      Buffers: shared hit=1098"
"      -> Seq Scan on publ (cost=0.00..500.61 rows=13461 width=145) (actual time=0.450..0.450 rows=13461 loops=1)
"        Buffers: shared hit=1098"
"Planning:"
"  Buffers: shared hit=150 read=1"
"Planning Time: 1.602 ms"
"Execution Time: 318.660 ms"
```

```
"Gather (cost=1668.87..44588.86 rows=1 width=129) (actual time=123.104..129.285 rows=0.00 loops=1)
"  Workers Planned: 2"
"  Workers Launched: 2"
"  Buffers: shared hit=15988 read=11909"
"  -> Hash Join (cost=668.87..43588.76 rows=1 width=129) (actual time=67.056..67.058 rows=0.00 loops=1)
"    Hash Cond: ((auth.pubid)::text = (publ.pubid)::text)"
"    Buffers: shared hit=15988 read=11909"
"    -> Parallel Seq Scan on auth (cost=0.00..42919.84 rows=10 width=23) (actual time=7.168..7.168 rows=10 loops=1)
"      Filter: ((name)::text = 'Divesh Srivastava'::text)"
"      Rows Removed by Filter: 1031673"
"      Buffers: shared hit=14890 read=11909"
"    -> Hash (cost=500.61..500.61 rows=13461 width=145) (actual time=4.462..4.463 rows=13461 loops=1)
"      Buckets: 16384 Batches: 1 Memory Usage: 2468kB"
"      Buffers: shared hit=1098"
"      -> Seq Scan on publ (cost=0.00..500.61 rows=13461 width=145) (actual time=0.449..0.449 rows=13461 loops=1)
"        Buffers: shared hit=1098"
"Planning Time: 0.191 ms"
"Execution Time: 129.636 ms"
```

unique non-clustering on Publ.pubID (Q1/Q2):

```
DROP INDEX IF EXISTS uniqueNonClusterIdxPubID;  
CREATE UNIQUE INDEX uniqueNonClusterIdxPubID on publ(pubID);  
EXPLAIN ANALYZE SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
DROP INDEX IF EXISTS uniqueNonClusterIdxPubID;  
CREATE UNIQUE INDEX uniqueNonClusterIdxPubID on publ(pubID);  
EXPLAIN ANALYZE SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

```
"Gather (cost=1668.87..49274.43 rows=45245 width=144) (actual time=4.472..273.805 rows=2324.00 loops=1)  
" Workers Planned: 2"  
" Workers Launched: 2"  
" Buffers: shared hit=16112 read=11785"  
" -> Hash Join (cost=668.87..43749.93 rows=18852 width=144) (actual time=62.506..193.706 rows=18852 loops=1)  
" Hash Cond: ((auth.pubid)::text = (publ.pubid)::text)"  
" Buffers: shared hit=16112 read=11785"  
" -> Parallel Seq Scan on auth (cost=0.00..39695.67 rows=1289667 width=38) (actual time=62.506..193.706 rows=18852 loops=1)  
" Buffers: shared hit=15014 read=11785"  
" -> Hash (cost=500.61..500.61 rows=13461 width=145) (actual time=5.745..5.746 rows=13461 loops=1)  
" Buckets: 16384 Batches: 1 Memory Usage: 2468kB"  
" Buffers: shared hit=1098"  
" -> Seq Scan on publ (cost=0.00..500.61 rows=13461 width=145) (actual time=0.457..0.458 rows=13461 loops=1)  
" Buffers: shared hit=1098"  
"Planning:"  
" Buffers: shared hit=9 read=4"  
"Planning Time: 1.379 ms"  
"Execution Time: 274.159 ms"
```

```
"Gather (cost=1000.28..43996.30 rows=1 width=129) (actual time=129.982..136.823 rows=0.00 loops=1)  
" Workers Planned: 2"  
" Workers Launched: 2"  
" Buffers: shared hit=15595 read=11572 written=1"  
" -> Nested Loop (cost=0.29..42996.20 rows=1 width=129) (actual time=66.466..66.467 rows=0.00 loops=1)  
" Buffers: shared hit=15595 read=11572 written=1"  
" -> Parallel Seq Scan on auth (cost=0.00..42919.84 rows=10 width=23) (actual time=5.337..5.338 rows=10 loops=1)  
" Filter: ((name)::text = 'Divesh Srivastava')::text)"  
" Rows Removed by Filter: 1031673"  
" Buffers: shared hit=15242 read=11557"  
" -> Index Scan using uniquenonclusteridxpubid on publ (cost=0.29..7.64 rows=1 width=145) (actual time=0.000..0.000 rows=1 loops=1)  
" Index Cond: ((pubid)::text = (auth.pubid)::text)"  
" Index Searches: 183"  
" Buffers: shared hit=353 read=15 written=1"  
"Planning:"  
" Buffers: shared hit=9 read=4"  
"Planning Time: 1.821 ms"  
"Execution Time: 136.852 ms"
```

clustering on Publ.pubID and Auth.pubID (Q1/Q2):

```
DROP INDEX IF EXISTS clusterIdxAuthPubID;
DROP INDEX IF EXISTS clusterIdxPublPubID;
CREATE INDEX clusterIdxPublPubID on publ(pubID);
CLUSTER publ USING clusterIdxPublPubID;
CREATE INDEX clusterIdxAuthPubID on auth(pubID);
CLUSTER auth USING clusterIdxAuthPubID;
Explain Analyze SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
DROP INDEX IF EXISTS clusterIdxAuthPubID;
DROP INDEX IF EXISTS clusterIdxPublPubID;
CREATE INDEX clusterIdxPublPubID on publ(pubID);
CLUSTER publ USING clusterIdxPublPubID;
CREATE INDEX clusterIdxAuthPubID on auth(pubID);
CLUSTER auth USING clusterIdxAuthPubID;
Explain Analyze SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

```
"Gather (cost=1670.87..50926.81 rows=45245 width=144) (actual time=7.108..238.837 rows=2324.00 loops=1)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=927 read=26987"
" -> Hash Join (cost=670.87..45402.31 rows=18852 width=144) (actual time=107.829..183.708 rows=18852 loops=1)
" Hash Cond: ((auth.pubid)::text = (publ.pubid)::text)"
" Buffers: shared hit=927 read=26987"
" -> Parallel Seq Scan on auth (cost=0.00..39706.67 rows=1289667 width=38) (actual time=107.829..183.708 rows=18852 loops=1)
" Buffers: shared hit=189 read=26621"
" -> Hash (cost=502.61..502.61 rows=13461 width=145) (actual time=5.502..5.503 rows=13461 loops=1)
" Buckets: 16384 Batches: 1 Memory Usage: 2468kB"
" Buffers: shared hit=738 read=366"
" -> Seq Scan on publ (cost=0.00..502.61 rows=13461 width=145) (actual time=0.228..0.228 rows=13461 loops=1)
" Buffers: shared hit=738 read=366"
"Planning:"
" Buffers: shared hit=12 read=16"
"Planning Time: 1.892 ms"
"Execution Time: 239.347 ms"
```

```
"Gather (cost=1000.28..44007.40 rows=1 width=129) (actual time=130.501..133.869 rows=0.00 loops=1)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=541 read=26636"
" -> Nested Loop (cost=0.29..43007.30 rows=1 width=129) (actual time=47.542..47.542 rows=0.00 loops=1)
" Buffers: shared hit=541 read=26636"
" -> Parallel Seq Scan on auth (cost=0.00..42930.84 rows=10 width=23) (actual time=7.084..7.084 rows=10 loops=1)
" Filter: ((name)::text = 'Divesh Srivastava'::text)"
" Rows Removed by Filter: 1031673"
" Buffers: shared hit=189 read=26621"
" -> Index Scan using clusteridxpublpubid on publ (cost=0.29..7.64 rows=1 width=145) (actual time=0.228..0.228 rows=1 loops=1)
" Index Cond: ((pubid)::text = (auth.pubid)::text)"
" Index Searches: 183"
" Buffers: shared hit=352 read=15"
"Planning:"
" Buffers: shared hit=13 read=15"
"Planning Time: 1.196 ms"
"Execution Time: 133.889 ms"
```

Discussion Discuss your observations. Is the choice of the strategy expected? How does the system come to this choice?

Die von PostgreSQL gewählten Join-Strategien entsprechen weitgehend den erwarteten Entscheidungen eines kostenbasierten Optimierers. Die Wahl der Strategie hängt hauptsächlich von der Größe der Relationen, der Selektivität der Prädikate sowie der Verfügbarkeit geeigneter Indizes ab.

Für Q1 verwendet PostgreSQL in allen Varianten einen Hash Join. Obwohl geeignete Indizes existieren, müssen große Teile der Tabelle `auth` verarbeitet werden. Der Query-Plan zeigt einen vollständigen Parallel Seq Scan auf `auth`, während `pub1` vollständig in eine Hash-Tabelle geladen wird. Da die Tabelle `pub1` relativ klein ist, kann die Hash-Tabelle vollständig im Hauptspeicher gehalten werden (`Batches: 1`). Dadurch entstehen keine zusätzlichen Kosten durch temporäre Dateien oder mehrfache Hash-Partitionierung. Der Hash Join ist in diesem Szenario effizient, da keine zusätzlichen Sortieroperationen notwendig sind und die Join-Bedingung direkt über Hash-Werte ausgewertet werden kann.

Die vorhandenen Indizes verbessern die Laufzeit von Q1 nur geringfügig. Der Hauptkostenfaktor bleibt das Lesen großer Teile der Tabelle `auth`. Die clustered Variante zeigt dennoch leichte Vorteile, da die physische Ordnung der Daten die Lokalität der Seitenzugriffe verbessert und dadurch sequentielle Zugriffe effizienter werden.

Für Q2 ist die Situation deutlich anders. Das Selektionsprädikat `name = 'Divesh Srivastava'` reduziert die Anzahl der relevanten Tupel aus `auth` stark. Ohne geeigneten Index verwendet PostgreSQL weiterhin einen Hash Join, da die Tabelle `auth` zunächst vollständig gescannt werden muss, um die passenden Tupel zu finden.

Sobald jedoch ein Index auf `pub1.pubID` vorhanden ist, wechselt PostgreSQL zu einem Indexed Nested Loop Join. Dies ist erwartbar, da nun nur wenige Tupel aus `auth` verarbeitet werden und für jedes Tupel ein effizienter Indexzugriff auf `pub1` möglich ist. Dadurch werden unnötige vollständige Tabellen-Scans auf der zweiten Relation vermieden.

Insgesamt zeigen die Ergebnisse deutlich, dass PostgreSQL die Join-Strategie anhand der geschätzten Kardinalitäten, der Selektivität der Prädikate sowie der erwarteten I/O-Kosten auswählt.

Indexed Nested Loop Join

Response times

Indexes	Response time Q1 [ms]	Response time Q2 [ms]
index on Publ.pubID	2718.627 ms	78.909 ms
index on Auth.pubID	54.444 ms	129.015 ms
index on Publ.pubID and Auth.pubID	55.605 ms	83.245 ms

Query plans

Index on Publ.pubID (Q1/Q2):

```
SET enable_nestloop to true;
SET enable_hashjoin to false;
SET enable_mergejoin to false;
DROP INDEX IF EXISTS nonClusterIdxPubl;
CREATE INDEX nonClusterIdxPubl on publ(pubID);
Explain Analyze SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
SET enable_nestloop to true;
SET enable_hashjoin to false;
SET enable_mergejoin to false;
DROP INDEX IF EXISTS nonClusterIdxPubl;
CREATE INDEX nonClusterIdxPubl on publ(pubID);
Explain Analyze SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

```
"Gather (cost=1000.28..448965.44 rows=45245 width=144) (actual time=0.342..2718.520 rows=2324.00)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=6193329 read=26209"
" -> Nested Loop (cost=0.29..443440.94 rows=18852 width=144) (actual time=1422.454..2693.326 r
" Buffers: shared hit=6193329 read=26209"
" -> Parallel Seq Scan on auth (cost=0.00..39706.67 rows=1289667 width=38) (actual time=
" Buffers: shared hit=659 read=26151"
" -> Index Scan using nonclusteridxpbl on publ (cost=0.29..0.30 rows=1 width=145) (actu
" Index Cond: ((pubid)::text = (auth.pubid)::text)"
" Index Searches: 3095201"
" Buffers: shared hit=6192670 read=58"
"Planning:"
" Buffers: shared hit=6 read=1"
"Planning Time: 0.614 ms"
"Execution Time: 2718.627 ms"
```

```
"Gather (cost=1000.28..44007.40 rows=1 width=129) (actual time=75.740..78.888 rows=0.00 loops=1)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=1293 read=25885"
" -> Nested Loop (cost=0.29..43007.30 rows=1 width=129) (actual time=58.294..58.295 rows=0.00)
" Buffers: shared hit=1293 read=25885"
" -> Parallel Seq Scan on auth (cost=0.00..42930.84 rows=10 width=23) (actual time=7.777
" Filter: ((name)::text = 'Divesh Srivastava'::text)"
" Rows Removed by Filter: 1031673"
" Buffers: shared hit=941 read=25869"
" -> Index Scan using nonclusteridxpbl on publ (cost=0.29..7.64 rows=1 width=145) (actu
" Index Cond: ((pubid)::text = (auth.pubid)::text)"
" Index Searches: 183"
" Buffers: shared hit=352 read=16"
"Planning:"
" Buffers: shared hit=7 read=1"
"Planning Time: 0.813 ms"
"Execution Time: 78.909 ms"
```

Index on Auth.pubID (Q1/Q2):

```
SET enable_nestloop to true;
SET enable_hashjoin to false;
SET enable_mergejoin to false;
DROP INDEX IF EXISTS nonClusterIdxPubl;
CREATE INDEX nonClusterIdxPubl on auth(pubID);
Explain Analyze SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
SET enable_nestloop to true;
SET enable_hashjoin to false;
SET enable_mergejoin to false;
DROP INDEX IF EXISTS nonClusterIdxPubl;
CREATE INDEX nonClusterIdxPubl on auth(pubID);
Explain Analyze SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

```
"Nested Loop (cost=0.43..87059.20 rows=45245 width=144) (actual time=0.065..54.365 rows=2324.00)
" Buffers: shared hit=40520 read=1726"
" -> Seq Scan on publ (cost=0.00..502.61 rows=13461 width=145) (actual time=0.012..0.817 rows=13461)
" Buffers: shared hit=368"
" -> Index Scan using nonclusteridxpbl on auth (cost=0.43..6.40 rows=3 width=38) (actual time=0.048..0.050 rows=3)
" Index Cond: ((pubid)::text = (publ.pubid)::text)"
" Index Searches: 13461"
" Buffers: shared hit=40152 read=1726"
"Planning:"
" Buffers: shared hit=13 read=1"
"Planning Time: 1.087 ms"
"Execution Time: 54.444 ms"
```

```
"Gather (cost=1000.00..46486.35 rows=1 width=129) (actual time=126.221..128.853 rows=0.00 loops=1)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=2935 read=24979"
" -> Nested Loop (cost=0.00..45486.25 rows=1 width=129) (actual time=110.365..110.366 rows=0.00)
" Join Filter: ((auth.pubid)::text = (publ.pubid)::text)"
" Rows Removed by Join Filter: 821121"
" Buffers: shared hit=2935 read=24979"
" -> Parallel Seq Scan on auth (cost=0.00..42930.84 rows=10 width=23) (actual time=6.366..6.367 rows=10)
" Filter: ((name)::text = 'Divesh Srivastava'::text)"
" Rows Removed by Filter: 1031673"
" Buffers: shared hit=1831 read=24979"
" -> Materialize (cost=0.00..569.91 rows=13461 width=145) (actual time=0.006..0.374 rows=13461)
" Storage: Memory Maximum Storage: 2410kB"
" Buffers: shared hit=1104"
" -> Seq Scan on publ (cost=0.00..502.61 rows=13461 width=145) (actual time=0.330..0.331 rows=13461)
" Buffers: shared hit=1104"
"Planning:"
" Buffers: shared hit=7 read=1"
"Planning Time: 0.703 ms"
"Execution Time: 129.015 ms"
```


Index on Auth.pubID and Auth.pubID (Q1/Q2):

```
SET enable_nestloop to true;
SET enable_hashjoin to false;
SET enable_mergejoin to false;
DROP INDEX IF EXISTS nonClusterIdxPubl;
DROP INDEX IF EXISTS nonClusterIdxAuth;
CREATE INDEX nonClusterIdxPubl on publ(pubID);
CREATE INDEX nonClusterIdxAuth on auth(pubID);
Explain Analyze SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
SET enable_nestloop to true;
SET enable_hashjoin to false;
SET enable_mergejoin to false;
DROP INDEX IF EXISTS nonClusterIdxPubl;
DROP INDEX IF EXISTS nonClusterIdxAuth;
CREATE INDEX nonClusterIdxPubl on publ(pubID);
CREATE INDEX nonClusterIdxAuth on auth(pubID);
Explain Analyze SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

```
"Nested Loop (cost=0.43..87059.20 rows=45245 width=144) (actual time=0.049..55.533 rows=2324.00)
" Buffers: shared hit=40564 read=1682"
" -> Seq Scan on publ (cost=0.00..502.61 rows=13461 width=145) (actual time=0.007..0.816 rows=13461)
" Buffers: shared hit=368"
" -> Index Scan using nonclusteridxauth on auth (cost=0.43..6.40 rows=3 width=38) (actual time=0.041..0.042 rows=3)
" Index Cond: ((pubid)::text = (publ.pubid)::text)"
" Index Searches: 13461"
" Buffers: shared hit=40196 read=1682"
"Planning:"
" Buffers: shared hit=13 read=2"
"Planning Time: 1.266 ms"
"Execution Time: 55.605 ms"
```

```
"Gather (cost=1000.28..44007.40 rows=1 width=129) (actual time=79.448..83.227 rows=0.00 loops=1)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=3029 read=24149"
" -> Nested Loop (cost=0.29..43007.30 rows=1 width=129) (actual time=62.449..62.453 rows=0.00)
" Buffers: shared hit=3029 read=24149"
" -> Parallel Seq Scan on auth (cost=0.00..42930.84 rows=10 width=23) (actual time=8.819..8.820 rows=10)
" Filter: ((name)::text = 'Divesh Srivastava'::text)"
" Rows Removed by Filter: 1031673"
" Buffers: shared hit=2677 read=24133"
" -> Index Scan using nonclusteridxpubl on publ (cost=0.29..7.64 rows=1 width=145) (actual time=0.000..0.001 rows=1)
" Index Cond: ((pubid)::text = (auth.pubid)::text)"
" Index Searches: 183"
" Buffers: shared hit=352 read=16"
"Planning:"
" Buffers: shared hit=14 read=2"
"Planning Time: 1.120 ms"
"Execution Time: 83.245 ms"
```

Discussion Discuss your observations. Are the response times expected? Why (not)?

Die gemessenen Laufzeiten des Indexed Nested Loop Join entsprechen den typischen Eigenschaften dieses Join-Verfahrens.

Für Q1 unterscheiden sich die Laufzeiten der verschiedenen Index-Konfigurationen deutlich. Besonders effizient ist die Variante mit einem Index auf `auth(pubID)`. PostgreSQL verwendet dabei die kleine Tabelle `publ` als äußere Relation und führt für jedes Tupel einen gezielten Indexzugriff auf `auth` aus. Da `publ` nur etwa 13 000 Tupel enthält, entstehen lediglich ungefähr 13 000 Index-Lookups. Dadurch bleibt die Laufzeit relativ gering.

Deutlich langsamer ist dagegen die Variante mit einem Index ausschließlich auf `publ(pubID)`. In diesem Fall verwendet PostgreSQL die große Tabelle `auth` als äußere Relation. Dadurch müssen mehr als drei Millionen Indexzugriffe auf `publ` durchgeführt werden (3095201 `Index Searches`). Obwohl jeder einzelne Zugriff effizient ist, führen die vielen wiederholten Lookups zu hohen I/O-Kosten und vielen zufälligen Seitenzugriffen. Deshalb steigt die Gesamtlaufzeit deutlich an.

Die Variante mit zwei Indizes liefert ähnliche Ergebnisse wie die Variante mit dem Index auf `auth(pubID)`, da PostgreSQL weiterhin den günstigsten Join-Pfad auswählt und hauptsächlich den effizienteren Zugriffspfad nutzt.

Für Q2 zeigt der Indexed Nested Loop Join seine typischen Stärken. Durch das hochselektive Prädikat auf `name` werden nur sehr wenige Tupel aus `auth` gefunden. Dadurch sind nur wenige Indexzugriffe auf die zweite Relation notwendig (183 `Index Searches`). In diesem Szenario ist der Indexed Nested Loop Join besonders effizient, da gezielte Index-Lookups wesentlich günstiger sind als vollständige Tabellen-Scans oder zusätzliche Sortieroperationen.

Insgesamt bestätigen die Ergebnisse die bekannten Eigenschaften des Indexed Nested Loop Join: sehr effizient bei kleinen oder hochselektiven Ergebnismengen, jedoch weniger geeignet für große Relationen mit sehr vielen benötigten Indexzugriffen.

Sort-Merge Join

Response times

Indexes	Response time Q1 [ms]	Response time Q2 [ms]
no index	812.562 ms	90.645 ms
two non-clustering indexes	335.165	71.343
two clustering indexes	364.270	236.776 ms

Query plans

No index (Q1/Q2):

```
SET enable_nestloop to false;
SET enable_hashjoin to false;
SET enable_mergejoin to true;
Explain Analyze SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
SET enable_nestloop to false;
SET enable_hashjoin to false;
SET enable_mergejoin to true;
Explain Analyze SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri

"Gather (cost=243556.43..254812.05 rows=45245 width=144) (actual time=556.592..801.204 rows=2324
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=4073 read=23851, temp read=37091 written=37153"
" -> Merge Join (cost=242556.43..249287.55 rows=18852 width=144) (actual time=647.158..758.430
" Merge Cond: ((auth.pubid)::text = (publ.pubid)::text)"
" Buffers: shared hit=4073 read=23851, temp read=37091 written=37153"
" -> Sort (cost=241130.63..244354.80 rows=1289667 width=38) (actual time=514.391..639.32
" Sort Key: auth.pubid"
" Sort Method: external merge Disk: 50680kB"
" Buffers: shared hit=2969 read=23851, temp read=37091 written=37153"
" Worker 0: Sort Method: external merge Disk: 49104kB"
" Worker 1: Sort Method: external merge Disk: 48656kB"
" -> Parallel Seq Scan on auth (cost=0.00..39706.67 rows=1289667 width=38) (actual
" Buffers: shared hit=2959 read=23851"
" -> Sort (cost=1425.80..1459.45 rows=13461 width=145) (actual time=4.987..5.358 rows=13
" Sort Key: publ.pubid"
" Sort Method: quicksort Memory: 2560kB"
" Buffers: shared hit=1104"
" Worker 0: Sort Method: quicksort Memory: 2560kB"
" Worker 1: Sort Method: quicksort Memory: 2560kB"
" -> Seq Scan on publ (cost=0.00..502.61 rows=13461 width=145) (actual time=0.275.
" Buffers: shared hit=1104"
"Planning:"
" Buffers: shared hit=27 dirtied=1"
"Planning Time: 0.748 ms"
"Execution Time: 812.562 ms"
```

```
"Gather (cost=45356.80..45424.27 rows=1 width=129) (actual time=87.153..90.456 rows=0.00 loops=1
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=4355 read=23569"
" -> Merge Join (cost=44356.80..44424.17 rows=1 width=129) (actual time=64.165..64.166 rows=0.
" Merge Cond: ((auth.pubid)::text = (publ.pubid)::text)"
" Buffers: shared hit=4355 read=23569"
" -> Sort (cost=42931.00..42931.03 rows=10 width=23) (actual time=56.786..56.789 rows=61
" Sort Key: auth.pubid"
" Sort Method: quicksort Memory: 25kB"
" Buffers: shared hit=3251 read=23569"
" Worker 0: Sort Method: quicksort Memory: 25kB"
" Worker 1: Sort Method: quicksort Memory: 25kB"
" -> Parallel Seq Scan on auth (cost=0.00..42930.84 rows=10 width=23) (actual time
" Filter: ((name)::text = 'Divesh Srivastava')::text)"
" Rows Removed by Filter: 1031673"
" Buffers: shared hit=3241 read=23569"
" -> Sort (cost=1425.80..1459.45 rows=13461 width=145) (actual time=4.987..5.281 rows=13
" Sort Key: publ.pubid"
```

```
"          Sort Method: quicksort  Memory: 2560kB"
"          Buffers: shared hit=1104"
"          Worker 0:  Sort Method: quicksort  Memory: 2560kB"
"          Worker 1:  Sort Method: quicksort  Memory: 2560kB"
"          -> Seq Scan on publ (cost=0.00..502.61 rows=13461 width=145) (actual time=0.358.
"              Buffers: shared hit=1104"
"Planning Time: 0.116 ms"
"Execution Time: 90.645 ms"
```

Two non-clustering indexes (Q1/Q2):

```
SET enable_nestloop = false;
SET enable_hashjoin = false;
SET enable_mergejoin = true;
DROP INDEX IF EXISTS nonClusterIdxPubl;
DROP INDEX IF EXISTS nonClusterIdxAuth;
CREATE INDEX nonClusterIdxPubl ON publ(pubID);
CREATE INDEX nonClusterIdxAuth ON auth(pubID);
Explain Analyze SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
SET enable_nestloop = false;
SET enable_hashjoin = false;
SET enable_mergejoin = true;
DROP INDEX IF EXISTS nonClusterIdxPubl;
DROP INDEX IF EXISTS nonClusterIdxAuth;
CREATE INDEX nonClusterIdxPubl ON publ(pubID);
CREATE INDEX nonClusterIdxAuth ON auth(pubID);
Explain Analyze SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

```
"Gather (cost=1000.72..111021.46 rows=45245 width=144) (actual time=0.203..335.091 rows=2324.00
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=10121 read=35972"
" -> Merge Join (cost=0.72..105496.96 rows=18852 width=144) (actual time=203.645..313.340 rows=18852)
" Merge Cond: ((auth.pubid)::text = (publ.pubid)::text)"
" Buffers: shared hit=10121 read=35972"
" -> Parallel Index Scan using nonclusteridxauth on auth (cost=0.43..101077.47 rows=12896)
" Index Searches: 1"
" Buffers: shared hit=8895 read=35915"
" -> Materialize (cost=0.29..1049.55 rows=13461 width=145) (actual time=0.041..4.313 rows=13461)
" Storage: Memory Maximum Storage: 17kB"
" Buffers: shared hit=1226 read=57"
" -> Index Scan using nonclusteridxpubl on publ (cost=0.29..1015.90 rows=13461 width=145)
" Index Searches: 3"
" Buffers: shared hit=1226 read=57"
"Planning:"
" Buffers: shared hit=18 read=10"
"Planning Time: 1.199 ms"
"Execution Time: 335.165 ms"
```

```
"Merge Join (cost=43934.07..44983.47 rows=1 width=129) (actual time=71.124..71.316 rows=0.00 loops=1)
" Merge Cond: ((publ.pubid)::text = (auth.pubid)::text)"
" Buffers: shared hit=12870 read=14358"
" -> Index Scan using nonclusteridxpubl on publ (cost=0.29..1015.90 rows=13461 width=145) (actual time=0.035..0.035 rows=1)
" Index Searches: 1"
" Buffers: shared hit=362 read=56"
" -> Sort (cost=43933.79..43933.85 rows=24 width=23) (actual time=67.341..67.539 rows=183.00 loops=1)
" Sort Key: auth.pubid"
" Sort Method: quicksort Memory: 25kB"
" Buffers: shared hit=12508 read=14302"
" -> Gather (cost=1000.00..43933.24 rows=24 width=23) (actual time=15.636..67.076 rows=183.00 loops=1)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=12508 read=14302"
" -> Parallel Seq Scan on auth (cost=0.00..42930.84 rows=10 width=23) (actual time=0.000..0.000 rows=10 loops=1)
" Filter: ((name)::text = 'Divesh Srivastava')::text)"
" Rows Removed by Filter: 1031673"
```

```
"          Buffers: shared hit=12508 read=14302"
"Planning:"
"  Buffers: shared hit=20 read=10"
"Planning Time: 1.511 ms"
"Execution Time: 71.343 ms"
```

Two clustering indexes (Q1/Q2):

```
SET enable_nestloop to false;
SET enable_hashjoin to false;
SET enable_mergejoin to true;
DROP INDEX IF EXISTS clusterIdxPublPubID;
DROP INDEX IF EXISTS clusterIdxAuthPubID;
CREATE INDEX clusterIdxPublPubID on publ(pubID);
CLUSTER publ USING clusterIdxPublPubID;
CREATE INDEX clusterIdxAuthPubID on auth(pubID);
CLUSTER auth USING clusterIdxAuthPubID;
Explain Analyze SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
SET enable_nestloop to false;
SET enable_hashjoin to false;
SET enable_mergejoin to true;
DROP INDEX IF EXISTS clusterIdxPublPubID;
DROP INDEX IF EXISTS clusterIdxAuthPubID;
CREATE INDEX clusterIdxPublPubID on publ(pubID);
CLUSTER publ USING clusterIdxPublPubID;
CREATE INDEX clusterIdxAuthPubID on auth(pubID);
CLUSTER auth USING clusterIdxAuthPubID;
Explain Analyze SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

"Gather (cost=1000.72..111021.46 rows=45245 width=144) (actual time=0.197..364.193 rows=2324.00
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=9710 read=36393 written=2"
" -> Merge Join (cost=0.72..105496.96 rows=18852 width=144) (actual time=220.493..339.898 rows=18852
" Merge Cond: ((auth.pubid)::text = (publ.pubid)::text)"
" Buffers: shared hit=9710 read=36393 written=2"
" -> Parallel Index Scan using clusteridxauthpubid on auth (cost=0.43..101077.47 rows=120000
" Index Searches: 1"
" Buffers: shared hit=8851 read=35969 written=2"
" -> Materialize (cost=0.29..1049.55 rows=13461 width=145) (actual time=0.026..6.405 rows=13461
" Storage: Memory Maximum Storage: 17kB"
" Buffers: shared hit=859 read=424"
" -> Index Scan using clusteridxpublpubid on publ (cost=0.29..1015.90 rows=13461 width=145)
" Index Searches: 3"
" Buffers: shared hit=859 read=424"
"Planning:"
" Buffers: shared hit=13 read=15"
"Planning Time: 1.314 ms"
"Execution Time: 364.270 ms"

"Merge Join (cost=43934.07..44983.47 rows=1 width=129) (actual time=236.719..236.748 rows=0.00 1
" Merge Cond: ((publ.pubid)::text = (auth.pubid)::text)"
" Buffers: shared hit=192 read=27036"
" -> Index Scan using clusteridxpublpubid on publ (cost=0.29..1015.90 rows=13461 width=145) (a
" Index Searches: 1"
" Buffers: shared hit=3 read=415"
" -> Sort (cost=43933.79..43933.85 rows=24 width=23) (actual time=230.942..230.979 rows=183.00
" Sort Key: auth.pubid"
" Sort Method: quicksort Memory: 25kB"
" Buffers: shared hit=189 read=26621"
" -> Gather (cost=1000.00..43933.24 rows=24 width=23) (actual time=27.999..230.665 rows=183.00
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=189 read=26621"
" -> Parallel Seq Scan on auth (cost=0.00..42930.84 rows=10 width=23) (actual time=27.999..230.665 rows=183.00


```
"          Filter: ((name)::text = 'Divesh Srivastava'::text)"
"          Rows Removed by Filter: 1031673"
"          Buffers: shared hit=189 read=26621"
"Planning:"
"  Buffers: shared hit=13 read=15"
"Planning Time: 1.889 ms"
"Execution Time: 236.776 ms"
```

Discussion Discuss your observations. Are the response times expected? Why (not)?

Die Ergebnisse des Sort-Merge Join entsprechen den erwarteten Eigenschaften dieses Join-Verfahrens. Merge Joins arbeiten besonders effizient, wenn beide Eingaberelationen bereits nach dem Join-Attribut sortiert vorliegen oder über geeignete Indizes in sortierter Reihenfolge gelesen werden können.

Für Q1 liefert die Variante ohne Indizes die schlechteste Laufzeit. Der Grund dafür ist, dass PostgreSQL beide Relationen zunächst sortieren muss. Besonders die große Tabelle `auth` verursacht hohe Kosten, da die Sortierung nicht vollständig im Hauptspeicher durchgeführt werden kann. Der Query-Plan zeigt deshalb einen `external merge` mit temporären Dateien auf der Festplatte. Diese zusätzlichen I/O-Operationen erhöhen die Laufzeit deutlich.

Die Varianten mit Indizes sind deutlich schneller. PostgreSQL kann die Tupel direkt über die B+-Baum-Indizes in sortierter Reihenfolge lesen und dadurch zusätzliche Sortieroperationen vermeiden. Besonders die zwei non-clustering Indizes liefern die beste Laufzeit, da beide Relationen effizient über Index Scans verarbeitet werden können.

Die clustered Variante verbessert die Laufzeit von Q1 ebenfalls, jedoch nicht stärker als die non-clustered Variante. Obwohl die physische Ordnung der Daten die Lokalität der Seitenzugriffe verbessert, müssen weiterhin große Teile beider Tabellen gelesen werden. Der zusätzliche Vorteil der physischen Sortierung bleibt daher begrenzt.

Für Q2 ist der Merge Join insgesamt weniger effizient als der Indexed Nested Loop Join. Obwohl nur wenige Ergebnistupel zurückgegeben werden, muss PostgreSQL weiterhin große Teile der Tabelle `auth` scannen, um die Selektionsbedingung auszuwerten. Anschließend müssen die relevanten Tupel nach `pubID` sortiert werden, bevor der eigentliche Merge Join durchgeführt werden kann.

Interessant ist, dass die clustered Variante bei Q2 deutlich langsamer als die non-clustered Variante ist. Der Hauptkostenfaktor bleibt der vollständige Parallel Seq Scan auf `auth`. Der Vorteil der physischen Ordnung kann in diesem Szenario kaum genutzt werden, da nur sehr wenige Tupel tatsächlich am Join teilnehmen. Dadurch überwiegen die zusätzlichen I/O-Kosten der großen Datenzugriffe.

Insgesamt zeigen die Ergebnisse deutlich, dass Merge Joins besonders gut für große, bereits sortierte Datenmengen geeignet sind. Bei kleinen oder hochselektiven Ergebnismengen sind Join-Verfahren mit gezielten Indexzugriffen meist effizienter.

Hash Join

Response times

Indexes	Response time Q1 [ms]	Response time [ms] Q2
no index	164.205 ms	79.341 ms

Query plans

No Index (Q1/Q2):

```
SET enable_nestloop to false;
SET enable_hashjoin to true;
SET enable_mergejoin to false;
Explain Analyze SELECT name, title FROM auth, publ WHERE auth.pubID = publ.pubID;
```

```
SET enable_nestloop to false;
SET enable_hashjoin to true;
SET enable_mergejoin to false;
Explain Analyze SELECT title FROM auth, publ WHERE auth.pubID = publ.pubID AND name = 'Divesh Sri
```

```
"Gather (cost=1670.87..50926.81 rows=45245 width=144) (actual time=2.885..163.935 rows=2324.00 loops=1)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=1568 read=26346"
" -> Hash Join (cost=670.87..45402.31 rows=18852 width=144) (actual time=72.944..143.180 rows=18852)
" Hash Cond: ((auth.pubid)::text = (publ.pubid)::text)"
" Buffers: shared hit=1568 read=26346"
" -> Parallel Seq Scan on auth (cost=0.00..39706.67 rows=1289667 width=38) (actual time=72.944..143.180 rows=18852)
" Buffers: shared hit=471 read=26339"
" -> Hash (cost=502.61..502.61 rows=13461 width=145) (actual time=5.674..5.674 rows=13461)
" Buckets: 16384 Batches: 1 Memory Usage: 2468kB"
" Buffers: shared hit=1097 read=7"
" -> Seq Scan on publ (cost=0.00..502.61 rows=13461 width=145) (actual time=1.001..1.001 rows=13461)
" Buffers: shared hit=1097 read=7"
```

"Planning:"

" Buffers: shared hit=14 dirtied=1"

"Planning Time: 0.693 ms"

"Execution Time: 164.205 ms"

```
"Gather (cost=1670.87..44601.86 rows=1 width=129) (actual time=75.695..78.984 rows=0.00 loops=1)
" Workers Planned: 2"
" Workers Launched: 2"
" Buffers: shared hit=1857 read=26057"
" -> Hash Join (cost=670.87..43601.76 rows=1 width=129) (actual time=60.068..60.070 rows=0.00 loops=1)
" Hash Cond: ((auth.pubid)::text = (publ.pubid)::text)"
" Buffers: shared hit=1857 read=26057"
" -> Parallel Seq Scan on auth (cost=0.00..42930.84 rows=10 width=23) (actual time=8.366..8.366 rows=10)
" Filter: ((name)::text = 'Divesh Srivastava'::text)"
" Rows Removed by Filter: 1031673"
" Buffers: shared hit=753 read=26057"
" -> Hash (cost=502.61..502.61 rows=13461 width=145) (actual time=3.714..3.715 rows=13461)
" Buckets: 16384 Batches: 1 Memory Usage: 2468kB"
" Buffers: shared hit=1104"
" -> Seq Scan on publ (cost=0.00..502.61 rows=13461 width=145) (actual time=0.413..0.413 rows=13461)
" Buffers: shared hit=1104"
```

"Planning Time: 0.062 ms"

"Execution Time: 79.341 ms"

Discussion What do you think about the response time of the hash index vs. the response times of sort-merge and index nested loop join for each of the queries? Explain.

Die Ergebnisse des Hash Join zeigen die typischen Eigenschaften dieses Join-Verfahrens sehr deutlich. Hash Joins sind besonders gut für große, unsortierte Datenmengen und Equality-Joins geeignet, da keine sortierten Eingaben oder vorhandenen Indizes benötigt werden.

Für Q1 liefert der Hash Join eine sehr gute Laufzeit und ist deutlich schneller als der Sort-Merge Join ohne geeignete Sortierung. PostgreSQL erstellt zunächst eine Hash-Tabelle für die kleinere Relation `pub1` und scannt anschließend die große Tabelle `auth`. Da die Tabelle `pub1` relativ klein ist, kann die komplette Hash-Tabelle vollständig im Hauptspeicher gehalten werden (`Batches: 1`). Dadurch entstehen keine zusätzlichen Kosten durch temporäre Dateien oder mehrfache Partitionierung der Hash-Tabelle.

Der Hash Join ist in diesem Szenario besonders effizient, da keine zusätzlichen Sortieroperationen notwendig sind und die Join-Bedingung direkt über Hash-Werte ausgewertet werden kann. Der Hauptkostenfaktor bleibt lediglich der vollständige Parallel Seq Scan auf `auth`.

Im Vergleich zum Indexed Nested Loop Join ist der Hash Join bei Q1 dennoch etwas langsamer als die beste Nested-Loop-Variante mit einem Index auf `auth(pubID)`. Dort kann PostgreSQL die kleine Tabelle `pub1` als äußere Relation verwenden und nur relativ wenige gezielte Indexzugriffe durchführen.

Für Q2 ist der Hash Join weniger effizient als der Indexed Nested Loop Join. Das Selektionsprädikat auf `name` ist hoch selektiv und liefert nur sehr wenige Tupel aus `auth`. In diesem Fall wäre ein gezielter Indexzugriff günstiger als das vollständige Scannen großer Teile der Tabellen und der Aufbau einer Hash-Tabelle.

Trotzdem bleibt der Hash Join bei Q2 schneller als der Sort-Merge Join ohne geeignete Sortierung, da keine zusätzlichen Sortieroperationen notwendig sind. PostgreSQL profitiert außerdem von der Parallelisierung des Seq Scans auf `auth`, wodurch die Selektionsbedingung effizient ausgewertet werden kann.

Insgesamt bestätigen die Ergebnisse die bekannten Eigenschaften von Hash Joins: sehr effizient für große unsortierte Relationen und Equality-Joins, jedoch weniger geeignet für hochselektive Anfragen mit nur wenigen Ergebnistupeln.

Time Spent on this Assignment

Time in hours per person: 10

References

Important: Reference your information sources!

Remove this section if you use footnotes to reference your information sources.

0.1 Drop All Indexes

```
DROP INDEX IF EXISTS uniqueNonClusterIdxPubID;  
DROP INDEX IF EXISTS clusterIdxAuthPubID;  
DROP INDEX IF EXISTS clusterIdxPublPubID;  
DROP INDEX IF EXISTS nonClusterIdxPubl;  
DROP INDEX IF EXISTS nonClusterIdxAuth;  
DROP INDEX IF EXISTS uniquenonclusteridxpubid;  
DROP INDEX IF EXISTS clusteridxpublpubid;  
DROP INDEX IF EXISTS clusteridxauthpubid;  
DROP INDEX IF EXISTS nonclusteridxpubl;  
DROP INDEX IF EXISTS nonclusteridxauth;
```